HW 3

All questions carry a weight of **25** points. TAs will evaluate using **10** sets of test data for each question, with each set of test data worth **2.5** points.

Note that the 2nd and 3rd questions will read/write files. You can put the example files and your main program file together such as in the same folder.

1. Combination:

You have a factorial function for n! as follows.

```
def factorial(x):
    if x == 1:
        return 1
    else:
        return (x * factorial(x-1))
```

Request:

I. Please use above factorial function to get the result of the combination C(n, k) or C_k^n, where k, n, and the result are positive integers, 0 < k < n, k n, and

$$C_k^n = \binom{n}{k} = \frac{P_k^n}{k!} = \frac{n!}{k!(n-k)!}$$

II. k and n are input from a user in one line with a space, and the order is k and n

III. You need to check whether 0 < k < n until user input correct k and n

IV. Your program CANNOT be interrupted by any input error. Here are the examples.

Case	Output
k > n	`Error! k > n, please input again!`
k = n	`Error! k = n, please input again!`
k <= 0	`Error! k <= 0, please input again!`
n <= 0	`Error! n <= 0, please input again!`

If there are multiple errors, please output errors with the order in the above form.

Test data

Example 1

input

```
6 49
```

output

```
13983816
```

Example 2

input

```
16 3
```

output

```
Error! k > n, please input again!
```

input

```
3 16
```

output

```
560
```

Example 3

input

```
2 3
```

output

```
3
```

Example 4

input

```
-3 -2
```

output

```
Error! k <= 0, please input again!
```

```
Error! n <= 0, please input again!
```

input

```
3 16
```

output

```
560
```

2. Extended Simple Calculator:

Right now, we extend the simple calculator in the practice and modify it as follows.

TAs will give you an input file name and an output file name. You need to let the calculator can read data from the input file, process the result based on the read data, one of the four fundamental operations of arithmetic, and writes the result to the output file, where the format of four fundamental operations of arithmetic is `number1 operator number2`, and number1, operator, and number2 are separated by spaces. Furthermore, number1, number2, and the calculated result may be float, and operators are only +, -, *, /. Handle exceptions related to file operations, format error in the file, and calculation error.

Here are some examples whose results are shown on the screen:

- no input file, output will be “File not found”
- operator is not +, -, *, /, output will be “Operator error”
- operator is / and `number2` is 0, output will be “Cannot divide by zero”
- formula is only `number1 number2`, output will be “Operator lost”
- formula is only `operator number2`, output will be “Number1 lost”
- formula is only `number1 operator`, output will be “Number2 lost”

If there is a calculated result, please output the whole calculated result to the file, no matter if the calculated result is integer or float, such as 2, 3.187343, and 78.63. If the data is successfully copied, you will show “Successfully” on the screen. Furthermore, no matter if there is any error or not, “Execution completed” will always be shown on the screen at the end of execution. The input and output file names will be written in one line and separated by a comma.

Note that if the result can be calculated, the content of the output file is the result; otherwise, there is no content in the output file as Example 3, i.e., the output file is empty.

Test data

Example 1

input

```
HW3-2-1.txt,out1.txt
```

output (on the screen)

```
Successfully
Execution completed
```

Content in HW3-2-1.txt

```
3 + 2
```

Result in out1.txt, which is generated by your code.

5

Example 2
input
```
HW3-2-2.txt,HW3-2-2-out.txt
```
output (on the screen)
```
Successfully
Execution completed
```

Content in HW3-2-2.txt
```
5 * 10
```
Result in HW3-2-2-out.txt, which is generated by your code.
```
50
```

Example 3
input
```
2-3.txt,out3.txt
```
output (on the screen)
```
File not found
Execution completed
```

There is no 2-3.txt

out3.txt is an empty file and generated by your code.

3. Calculate and Search:

TAs will give you a text file's name with .txt. You will read this txt file with encoding utf-8, and in the file, there are `N` students and their scores of three objects. Moreover, the scores are integers and not smaller than 0.

In the txt file, the first line shows the number of `N`, where 1 <= `N` <= 100. From 2 to `N+1` lines are the students' numbers and their scores and the order of columns is "No. object1's score object2's score object3's score", separated by a space. You can refer to the following examples for more details.

Please read the file to calculate the total score for each student, find the highest total score among them, and print the No. of student and corresponding total score in one line and separated by comma.

Note: there is only one highest total score in the file.

Test data

Example 1

input

```
score.txt
```

output

```
4,254
```

Contents in score.txt

```
4
1 70 73 62
2 63 90 92
3 73 68 60
```

```
4 83 73 98
```

Example 2
input
```
3-2.txt
```
output
```
8,269
```

Contents in 3-2.txt
```
10
1 70 73 62
2 63 90 92
3 73 68 60
4 83 73 98
5 61 99 67
6 79 75 72
7 62 77 84
8 91 80 98
9 100 70 68
10 79 84 85
```

4. Loop and Condition:

Given two numbers x and y by one input function, within one line and separated by space, like 5 12, where x is the first number of a series of numbers, y is the location of a series of numbers, and x and y are positive integers.

The pattern of generating the next number is as follows.

- If the number is odd, multiply it by 5 and then minus it by 3.
- If the number is even, divide it by 2 and then add it by 1.
- Repeat above two rules until the location is y.
- Print the number in the location y.

Suppose that TAs will give correct numbers of x and y. You do NOT need to check the x and y.

For example, x and y are 5 and 12.
Following the above patterns, the completed process is:
5 22 12 7 32 17 82 42 22 12 7 32
x y
1 2 3 … 12
Therefore, the result will be 32.

Hint: location is different to the "index" in Python. For the relationship between location and index is that the value of location is the value of index plus 1, i.e., location = index + 1

Test data

Example 1

input

```
5  12
```

output

```
32
```

Example 2

input

```
1  1
```

output

```
1
```

Example 3

input

```
54  3
```

output

```
15
```

Example 4

input

```
2  9
```

output

```
2
```